Lesson 14 Teacher's Key: Raising the Stakes

TIC-80 - Instructor Reference Only

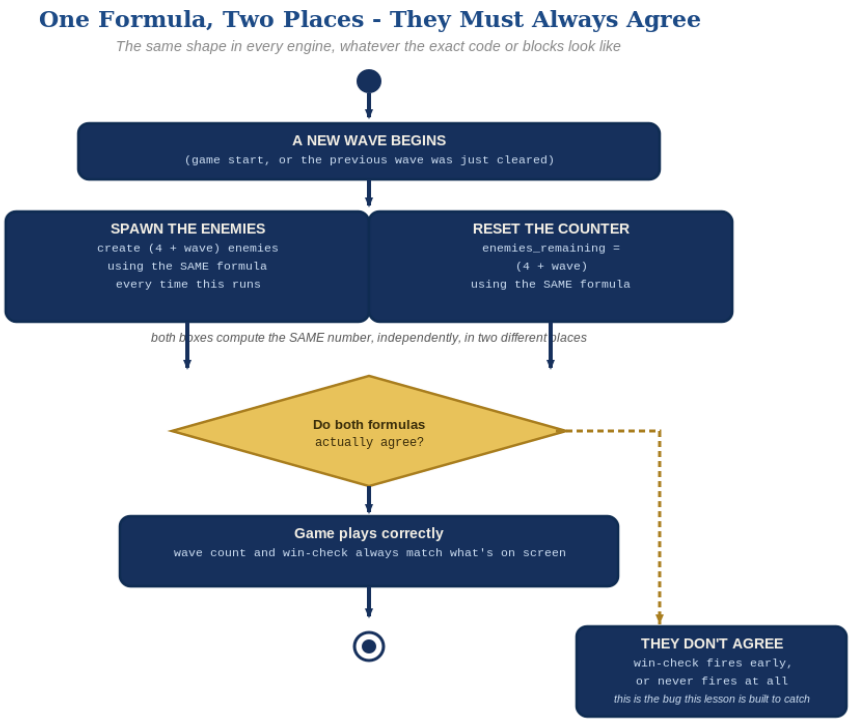
VIDEO GAME CREATION

What This Lesson Teaches

Same idea as every engine this week - replace hardcoded literals with wave-dependent formulas - expressed as plain Lua inside spawn_wave(), introduced in Lesson 11.

Level 1 - Full Worked Solution

- local count = 4 + wave computes the count once, then reuses it for both the spawn loop and enemies_remaining - a single source of truth by construction.
- No new fields are needed for Level 1 count scaling - only Level 2's speed scaling adds a new field (speed) to each enemy's table entry.
- Wave 1 still spawns exactly 5 (4 + 1 = 5), so nothing about the game's opening feel changes - only later waves escalate.



Whether it's two places (most engines) or the loop and the reset living in genuinely different scripts (Godot, GameMaker), the rule is the same: one formula, referenced everywhere.

VIDEO GAME CREATION

Whether the spawn loop and the reset live in two places or two genuinely different scripts, the rule is the same: one formula, referenced everywhere.

Why This Structure

- Storing count in a local variable (rather than repeating the formula) guarantees the spawn loop and enemies_remaining can never disagree - the same discipline as every other engine this week, in plain Lua.
- Adding a speed field to each enemy's own table entry (rather than a single global speed) keeps each enemy's full state - position, alive-ness, flash status from Lesson 13, and now speed - together in one place, the same discipline established since Lesson 4.

Level 2 / Level 3 Talking Points

- Ask students to trace spawn_wave() for wave 3 by hand: count = 4 + 3 = 7 - a good comprehension check.
- For students comparing engines: point out that GameMaker's identical (4 + wave) formula, expressed as GML instead of Lua, is conceptually the exact same idea - both engines needed a new per-instance/per-entry speed value where none existed before.

Common Student Mistakes

- Updating the spawn loop's count but forgetting to update enemies_remaining (or vice versa) - avoided entirely if count is stored in a local variable and reused.

- Setting speed on the table entry AFTER table.insert() instead of as part of the inserted table itself, which can silently fail depending on how the insert is written.
- Forgetting to change the movement code to read e.speed instead of a leftover literal - the field exists but is never actually used.